

Week 10

2. UNIX Pipes (I) – Examples

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Wednesday, November 5th, 2025



University of Windsor

Content

Midterm Exam Review

Question 3

Question 5

Question 6

2

Unnamed pipes

The `pipe()` system call

Rules

Example

The `dup2()` system call

Examples

Part I

Unnamed Pipes – Review



Unnamed pipes

Unnamed pipes

- (Unnamed) Pipes (vs FIFO), known as `pipe`, are a mechanism for Inter-Process Communication (IPC).
- Pipes are used by shells to connect one utility's standard output with the standard input of another utility.
- Example: `ps -ef | grep bash | wc -l`
- A process creates a pipe, forks then, uses the pipe to exchange information with its child.

Limitations

Pipes are the oldest form of Unix IPC. They have two limitations:

- They are **half-duplex**: data flows in one direction only.
- They can be used only between processes that have a common ancestor.

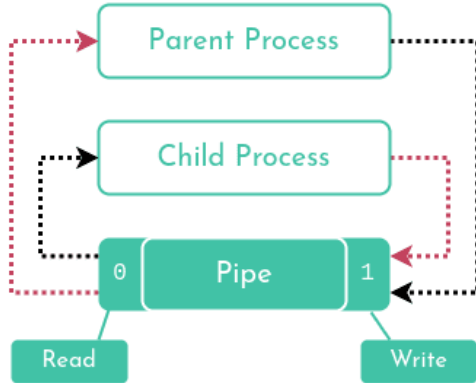
Pipes

A pipe is a connection between two processes, such that the standard output from one process becomes the standard input of the other process. In UNIX Operating System, Pipes are useful for communication between related processes (Inter-Process Communication – IPC):

- Pipe is one-way communication only. i.e., we can use a pipe such that one process writes to the pipe, and the other process reads from the pipe. It opens a pipe, which is an area of main memory that is treated as a **"virtual file"**.
- The pipe can be used by the creating process, as well as all its child processes, for reading and writing. One process can write to this "virtual file" or pipe and another related process can read from it.
- If a process tries to read before something is written to the pipe, the process is suspended until something is written.
- The pipe system call finds the first two available positions in the process's open file table and allocates them for the read and write ends of the pipe



Pipes



such that the standard output of the other process. In UNIX communication between related

can use a pipe such that One process reads from the pipe. It opens a file treated as a "*virtual file*".

, as well as all its child processes, refer to this "virtual file" or pipe and

written to the pipe, the process is

able positions in the process's open file table. The read and write ends of the pipe



The `pipe()` system call

`pipe()`

Synopsis: `int pipe(int fd[2])`

returns 0 when successful and -1 otherwise

`pipe()` creates a pipe and returns **two file descriptors**:

`fd[0]` and `fd[1]`

`fd[0]` is open for **reading**,

`fd[1]` is open for **writing**.

A **pipe** is a **one-way communication** channel between two related processes.

Rules when reading from and writing to a pipe

Rules

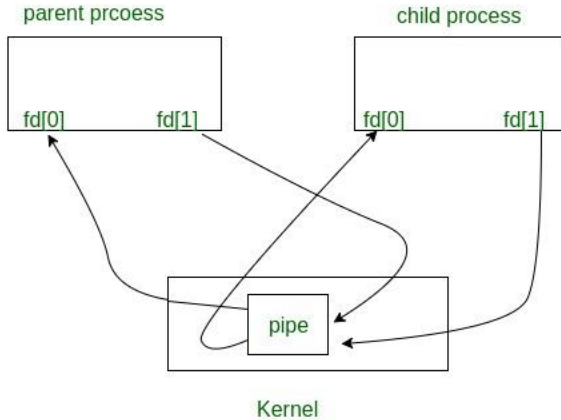
When reading from or writing to a pipe, the following rules apply:

1. If a process reads from a pipe whose write-end has been closed, after all data has been read, `read()` returns 0.
2. If a process reads from an empty pipe whose write-end is still open, it sleeps until some input becomes available.
3. If a process tries to read from a pipe more bytes than are present, `read()` reads all available bytes and returns the number of bytes read.
4. If a process writes to a pipe whose read-end has been closed, the write operation fails and the writer process receives a `SIGPIPE`.

In case of multiple processes writing to the same pipe, a write of up to `PIPE_BUF` bytes is guaranteed to be atomic.

→ data from different writer processes will not be interleaved.

fork () and Pipes



cross the child process.

cross parent process.

then the parent and child



● `fork()` and Pipes

○ `fork()` in any process:

- The file descriptors remain open across the child process.
- The file descriptors remain open across parent process.
- If we call `fork()` after creating a pipe, then the parent and child can communicate via the pipe.



Part II

Unnamed Pipes – Examples



○ Example 1: `pipe_ex1.c`



Example 1: Child is a writer, Parent is a reader...(main) (pipe_ex1.c)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

//please read these manual pages
//man 7 pipe , a very good read,
//man read
//man 2 write

void child(int *);
void parent(int *);

int main(int argc, char *argv[]){
    int fd[2];

    if(pipe(fd) == -1)
        exit(1);

    if(fork() == 0)
        child(fd);
    else
        parent(fd);
    exit(0);
}
```

Example 1: Child is a writer, Parent is a reader... (parent)

```
//parent is the reader
void parent(int *fd){
    char ch;

    close(fd[1]);
    printf("I am the parent process. The child process has sent the
message:\n");

    do{
        read(fd[0], &ch, 1);
        printf("%c", ch);
        if(ch == '\n')
            break;
    }while(1);
}
```

Example 1: Child is a writer, Parent is a reader... (child)

```
//child is the writer
void child(int *fd){

    char message[255]="Hello, here is my data...\n";

    close(fd[0]);
    write(fd[1], message, 26);
}
```

○ Example 2: pipe1.c



Example 2: Pipe1.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

#define MSGSIZE 16

char *msg1 = "hello, world #1";
char *msg2 = "hello, world #2";
char *msg3 = "hello, world #3";

//a pipe for one process only
int main() {
    char inbuf[MSGSIZE];
    int p[2], j;

    if (pipe(p) == -1) {
        perror("pipe call");
        exit(1);
    }

    write(p[1], msg1, MSGSIZE);
    write(p[1], msg2, MSGSIZE);
    write(p[1], msg3, MSGSIZE);

    for (j = 0; j < 3; j++) { //change 3 to 5
        read(p[0], inbuf, MSGSIZE);
        printf("%s\n", inbuf);
    }

    exit(0);
}
```

○ Example 3: `pipe2.c`



Example 3: Pipe2.c

```
//*****
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>

#define MSGSIZE 16

char *msg1 = "hello, world #1";
char *msg2 = "hello, world #2";
char *msg3 = "hello, world #3";

int main() {
    char inbuf[MSGSIZE];
    int p[2], j;
    pid_t pid;

    if (pipe(p) == -1) {
        perror("pipe call error!");
        exit(1);
    }

    switch (pid = fork()) {
        case -1:
            perror("fork call");
            exit(2);

        case 0: //child process
            write(p[1], msg1, MSGSIZE);

            write(p[1], msg2, MSGSIZE);

            write(p[1], msg3, MSGSIZE);
            break;

        default: //parent
            for (j = 0; j < 3; j++) { //change 3 to 5 again
                read(p[0], inbuf, MSGSIZE);
                printf("%s\n", inbuf);
            }

            wait(NULL);
    }
    exit(0);
}
```

○ Example 4: `pipe3.c`



Example 4: Pipe3.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>

#define MSGSIZE 16

char *msg1 = "hello, world #1";
char *msg2 = "hello, world #2";
char *msg3 = "hello, world #3";

int main() {
    char inbuf[MSGSIZE];
    int p[2], j;
    pid_t pid;

    if (pipe(p) == -1) {
        perror("pipe call");
        exit(1);
    }

    switch (pid = fork()) {
        case -1:
            perror("fork call");
            exit(2);
        case 0: //child process
            close(p[0]);
            write(p[1], msg1, MSGSIZE);
            write(p[1], msg2, MSGSIZE);
            write(p[1], msg3, MSGSIZE);
            break;

        default:
            close(p[1]);
            for (j = 0; j < 3; j++)
            {
                read(p[0], inbuf, MSGSIZE);
                printf("%s\n", inbuf);
            }
            wait(NULL);
    }
    exit(0);
}
```

○ Example 5: `pipesize.c`



Example 5: pipesize.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>

int count;
void alrm_action(int);

int main(){
    int p[2];
    int pipe_size;
    char c = 'x';
    static struct sigaction act;

    act.sa_handler = alrm_action;
    sigfillset(&act.sa_mask);

    if (pipe(p) == -1){
        perror("pipe call");
        exit(1);
    }

    pipe_size = fpathconf(p[0], _PC_PIPE_BUF);
    printf("Maximum size of write to pipe: %d bytes\n", pipe_size);

    sigaction(SIGALRM, &act, NULL);
    while (1){
        alarm(20);
        write(p[1], &c, 1);
        alarm(0);
        if (++count % 1024 == 0)
            printf("%d characters in pipe\n", count);
    }
}

void alrm_action(int signo)
{
    printf("write blocked after %d characters\n", count);
    exit(0);
}
```

○ Example 6: game .c



Example 6: 2-player game with a referee...(main) (game.c)

```
int main(int argc, char *argv[]){ int
    fd1[2], fd2[2], fd3[2], fd4[2]; char
    turn='T';

    printf("This is a 2-player game with a referee\n");
    pipe(fd1);
    pipe(fd2);
    if(!fork())
        player("TOTO", fd1, fd2);

    close(fd1[0]); // parent only write to pipe 1
    close(fd2[1]); // parent only reads from pipe 2

    pipe(fd3);
    pipe(fd4);
    if(!fork())
        player("TITI", fd3, fd4);

    close(fd3[0]); // parent only write to pipe 3
    close(fd4[1]); // parent only reads from pipe 4

    while(1){
        printf("\nReferee: TOTO plays\n\n");
        write(fd1[1], &turn, 1);
        read(fd2[0], &turn, 1);

        printf("\nReferee: TITI plays\n\n");
        write(fd3[1], &turn, 1);
        read(fd4[0], &turn, 1);
    }
}
```

Example 6: 2-player game with a referee...(player)

```
void player(char *s, int *fd1, int *fd2){
    int points=0;
    int dice;
    long int ss=0;
    char turn;
    close(fd1[1]);
    close(fd2[0]);
    while(1){
        read(fd1[0], &turn, 1);
        printf("%s: playing my dice\n", s);
        dice =(int) time(&ss)%10 + 1;
        printf("%s: got %d points\n", s, dice);
        points+=dice;
        printf("%s: Total so far %d\n\n", s, points);
        if(points >= 50){
            printf("%s: game over I won\n", s);
            kill(0, SIGTERM);
        }
        sleep(5);    // to slow down the execution
        write(fd2[1], &turn, 1);
    }
}
```

701968234901768023405879312016
670980321768767



774862586034187069210783468019
701968234901768023405879312016
670980321768767

WPOYFXI LFJXI G I E
H I MPHLOM ZFOLI PG
BXNYSXWHYGI JLG I L
OLI EPC OOUJPL QDZC
NYDWIGL OUPJL QDZC

Thank you!
Questions?

DEFYXHQGRUOSGRP RN	3236343972474116
QJRMVQJXVRI DKUEVP	893833239594578
KGWBGQFHP RSUP RU	0310019701552750
XURNYUXZ DCSYULM I	7875465400196596
WHMFSP UIQYSUQYRSHRG	5019353606937419

00110011100111
11111010000100
11000000001111
00111100111100
00101111111100
00000111011011
11001100111100

Dr. Andreas S. Maniatis
Assistant Professor
Andreas.Maniatis@UWindsor.ca
<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor